

```

    names: string[] = ['paul', 'david', 'ean', 'renee',
    'chloe']}]

```

The above code should produce the outputs 'void' and '['david', 'ean', 'renee']'.

Async pipe

This is one of the very interesting pipes. It works as an observer to a promise or an observable. If the promise is resolved or the observable emits a new value, it updates the view with the appropriate value(s). This can have a dual impact of reducing component-level code while also providing incredible performance optimization opportunities when utilized correctly.

The main benefit to utilizing the async pipe is not the reduction in code, but the performance optimization and data flow patterns that this feature can help enable. In Angular 2, through the `ChangeDetectionStrategy @Component` property, we are able to supply the framework with information regarding how change detection for a component should be handled.

Assuming our application state is engineered correctly, we can effectively turn off change detection within our application until it is absolutely necessary. The performance implications this has over a standard, two way data-binding type approach are significant.

```

import {Component, View} from 'angular2/core';
@Component({selector: 'async-pipe',})
@View({template: `
    <h2>Async Pipe Example</h2>
    <p>wait... {{promise | async}}</p>
    <button (click)="clickMe()">Click me to initiate
promise</button>`})
export class AsyncPipe {
    promise : Promise <string> = null;
    clickMe() {
        this.promise = new Promise<string>((resolve,
reject) => {
            setTimeout(function ()
{resolve("resolved");},2000);
        });}}

```

Pure and Impure pipes

Pipe also accepts a metadata called pure. All the predefined pipes discussed